INSTITUTO FEDERAL
Catarinense
Campus Camboriú

# Desenvolvimento Web I

**Professor: Euclides Paim**

*euclides.paim@ifc.edu.br*

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

# *Fundamentos de JavaScript*
# (Objetos)

**Professor: Euclides Paim**

*euclides.paim@ifc.edu.br*

# Desenvolvimento Web

*Introdução*

- **O que estudamos até agora?**
  - HTML
    - Elementos;
    - Atributos;
    - Imagens, tabelas, listas;
    - Classes;
    - Ids.
  - CSS
    - Sintaxe CSS;
    - Seletores;
    - *Box model*, *backgrounds*, *margins* e *padding,*
    - Posicionamento, transições e animações...
  - JS
    - Sintaxe
    - Variáveis
    - Operadores
    - Tipos de Dados
    - Funções

# Desenvolvimento Web I

*Sumário*

- **Objetos JavaScript**
  - Definição
  - Criação
    - Usando um literal
    - Através da palavra **`new`**
    - Através de um construtor
  - Propriedades do Objeto
  - Métodos do Objeto
  - *Getters and Setters*
  - Construtores

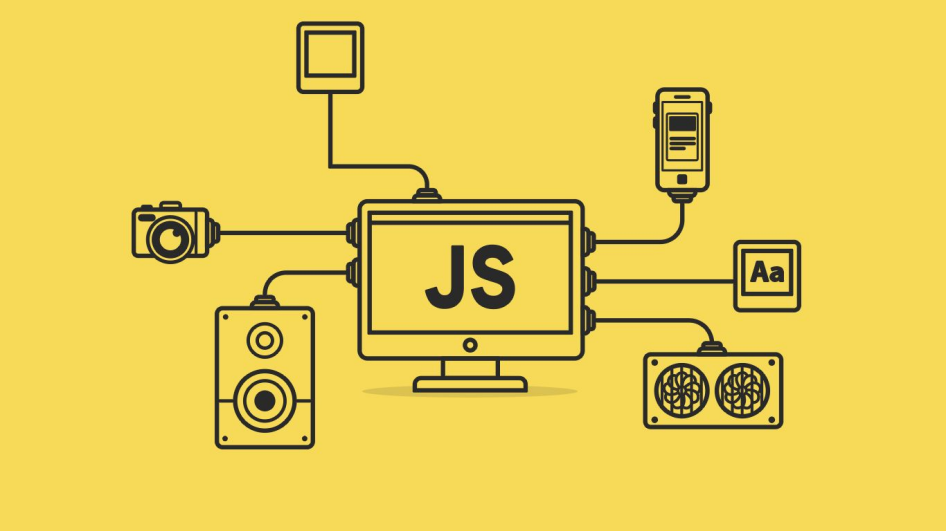
# Desenvolvimento Web

*Introdução*

- **Destaques importantes**
  - **JS**
    - A palavra-chave ***this***
      **=>** https://www.w3schools.com/js/js_this.asp
    - O que é ***hoisting***
      **=>** https://www.w3schools.com/js/js_hoisting.asp
    - O operador condicional **ternário**:
      **=>** https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Operators/Conditional_Operator
    - Métodos para obter **datas:**
      **=>** https://www.w3schools.com/js/js_date_methods.asp
    - Tipos de **escopos:**
      **=>** https://www.w3schools.com/js/js_scope.asp
    - Uso de **JSON:**
      **=>** https://www.w3schools.com/js/js_json.asp

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

# Desenvolvimento Web I



- **Visão Geral**

  - O tipo de dados fundamental do JavaScript é o **objeto**. Um objeto é um valor composto: agrega vários valores (valores primitivos ou outros objetos) e **permite armazenar e recuperar** esses valores por nome. Em JavaScript, quase "tudo" são um objeto:

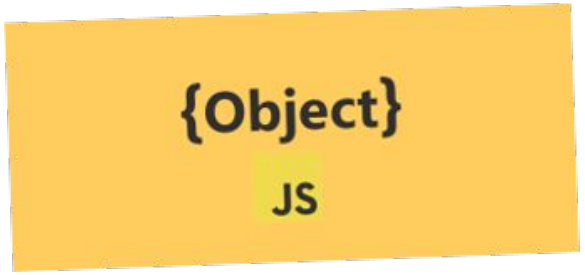
    - Booleanos podem ser objetos (se definidos com a palavra-chave **`new`**);
    - Os números podem ser objetos (se definidos com a palavra-chave **`new`**);
    - *Strings* podem ser objetos (se definidos com a palavra-chave **`new`**);
    - *Datas* são sempre objetos;
    - As matemáticas são sempre objetos;
    - Expressões regulares são sempre objetos;
    - *Arrays* são sempre objetos;
    - Funções são sempre objetos;
    - Objetos são sempre objetos.

- **Visão Geral**
  - Lembre-se que os objetos são mutáveis e são manipulados por referência ao invés de por valor. As coisas mais comuns a fazer com os objetos são criá-los e definir, consultar, excluir, testar e enumerar suas propriedades.
  - A maneira mais fácil de criar um objeto é incluir um **literal de objeto** no seu código JavaScript. Um literal de objeto é uma lista separada por vírgula de pares **nome: valor**, colocados entre chaves.

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

  - Em outras palavras, um objeto JavaScript é uma coleção de **valores nomeados.**

- **Existem diferentes maneiras de criar novos objetos**:
  - Definir e criar um único objeto, usando um **literal de objeto.**
  - Definir e criar um único objeto, com a palavra-chave **`new`**.
  - Definir um **construtor** de objetos e criar objetos do tipo construído.



- **Usando um literal de objeto**
  - Essa é a maneira mais fácil de criar um objeto JavaScript.
  - Usando um literal de objeto, definimos e criamos um objeto em uma instrução.
  - Um literal de objeto é uma lista de pares nome: valor (como idade: 50) dentro de chaves {}.
  - O exemplo a seguir cria um novo objeto JavaScript com quatro propriedades:

```
var person = {
  firstName:"John",
  lastName:"Doe",
  age:50,
  eyeColor:"blue"
};
```

- **Usando a palavra-chave *new***
  - O exemplo a seguir também cria um novo objeto JavaScript com quatro propriedades:

```
var person = new Object();
  person.firstName:"John",
  person.lastName:"Doe",
  person.age:50,
  person.eyeColor:"blue"
```

  - Considerando que os dois exemplos fazem exatamente o mesmo. Não há necessidade de usar `new Object()`.
  - Para simplicidade, legibilidade e velocidade de execução, use o método literal do objeto.

- **Objetos são mutáveis**
  - Objetos são endereçados por referência, não por valor. Se `person` for um objeto, a seguinte declaração não criará uma cópia de `person`:

```
// Esta declaração não vai criar um cópia de person.
const x = person;
```

  - O objeto `x` não é uma cópia de `person`. Ele é `person`. `X` e `person` são o mesmo objeto. Quaisquer alterações em `x` também mudarão em `person`, porque `x` e `person` são o **mesmo** objeto.

```
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"}

var x = person;
x.age = 10;            // Isto vai mudar ambos x.age e person.age
```

- **Propriedade do Objeto**
  - As **propriedades** são a parte mais importante de qualquer objeto JavaScript.
  - Propriedades são os valores associados a um objeto JavaScript.
  - Um objeto JavaScript é uma coleção de propriedades não ordenadas.
  - As propriedades geralmente podem ser alteradas, adicionadas e excluídas, mas algumas são somente leitura. A sintaxe para acessar a propriedade de um objeto é:

```
ObjectName.propriedade          // person.age

ObjectName.["propriedade"]      // person.["age"]

objectName.[expressão]          // x = "age"; person[x]
```

- **Propriedade do Objeto**
  - Os valores nomeados, nos objetos JavaScript, são chamados de **propriedades** .

| Propriedade | Valor |
|---|---|
| firstName | John |
| lastName | Doe |
| age | 50 |
| eyeColor | blue |

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

- **Propriedade do Objeto**
  - Podemos **adicionar** novas propriedades a um objeto existente, simplesmente atribuindo-lhe um valor.
  - Suponhamos que o objeto `person` já exista, podemos atribuir novas propriedades a ele.

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};

person.nationality = "English";
```

  - A palavra-chave **`delete`** exclui uma propriedade de um objeto:

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};

delete person.age;    // ou delete person["age"];
```

- **Métodos do Objeto**
  - Métodos são **ações** que podem ser executadas em objetos.
  - As propriedades do objeto podem ser valores primitivos, outros objetos e funções.
  - Um **método de objeto** é uma propriedade de objeto que contém uma **definição de função.**

| Propriedade | Valor |
|---|---|
| firstName | John |
| lastName | Doe |
| age | 50 |
| eyeColor | blue |
| fullName | function() {return this.firstName + " " + this.lastName;} |

Podemos dizer então que **Objetos JavaScript** são "**contêineres**" para valores nomeados, chamados **propriedades** e **métodos**.

- **Métodos do Objeto**
  - **Acessamos** um método do objeto com a seguinte sintaxe:

```
objectName.methodName()
```

  - Normalmente vamos declarar `fullName()` como um método do objeto `person` e
  - `fullName` como uma propriedade do mesmo objeto.
  - A propriedade *fullName* será executada (como uma função) quando for chamada com **()** .
  - Este exemplo acessa o **método** *fullName()* do objeto `person`:

```
name = person.fullName();
```

- **Métodos do Objeto**
  - **Adicionar** um método a objeto é feito com a seguinte sintaxe:

```
person.name = function () {
  return this.firstName + " " + this.lastName;
};
```

  - Estamos adicionando o método `name` ao objeto `person`;
  - Quando invocado o método retornará a concatenação das propriedades `firstName` e `lastName`.

Observe o uso da *keyword* **`this`** que aponta para o "proprietário" do método em que ela está inserida, neste caso o objeto `person`.

- ***Getters* e *setters***
  - Os ***Getters*** e ***setters*** são comuns em várias linguagens, no JavaScript não seria diferente, eles nos ajudam a encapsular/proteger/isolar propriedades e facilitar o trabalho com objetos.
  - Normalmente utilizamos quando precisamos fazer validações ou tratamentos antes de salvar um dado. O mesmo acontece para recuperá-los. Sem os *getters* e *setters* poderíamos fazer desta forma:

```
//definimos um valor
var = 123;

//resgatamos o valor para mostrar na tela
console.log( var.toFixed(2).replace(".",",") );
```

- ***Getters* e *setters***
  - Porém essa não é a forma mais elegante, para melhorar essa semântica e facilitar nossa vida foram incluídos os manipuladores `get` e `set` no *ECMAScript* 5.
  - A melhor forma de fazermos isso é implementando os manipuladores para tratar os valores dentro do objeto, a utilização é bem simples basta incluir o prefixo antes da função.

```
const var = {
  valor: 0,

  get moeda() { // define o get moeda
    return "R$ " + this.valor.toFixed(2).replace(".",",");
  }
  set int(i)
    return this.valor = i;
}
```

Leiura complementar.
Getters:https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Functions/get
Setters: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Functions/set

- ***Construtores***
  - Os exemplos dos capítulos anteriores são limitados. Eles criam apenas objetos únicos. Às vezes, precisamos de um " ***blueprint*** " para criar muitos objetos do mesmo "tipo".
  - A maneira de criar um "tipo de objeto" é usar uma **função construtora de objetos** . No exemplo abaixo, `function Person()` é uma função construtora de objetos.

```
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}
```

- ***Construtores***
  - Objetos do mesmo tipo são criados chamando a função construtora com a palavra-chave `new`, exemplo:

```
var myFather = new Person("John", "Doe", 50, "blue");

var myMother = new Person("Sally", "Rally", 48, "green");
```

# Desenvolvimento Web

*Resumo*

- **Objetos JavaScript**
  - Definição
  - Criação
    - Usando um literal
    - Através da palavra **`new`**
    - Através de um construtor
  - Propriedades do Objeto
  - Métodos do Objeto
  - *Getters and Setters*
  - Construtores

INSTITUTO FEDERAL
Catarinense
Campus Camboriú

# Referências


## Referências Básicas

SILVA, Maurício Samy. CSS3: desenvolva aplicações web profissionais com uso dos poderosos recursos de estilização das CSS3. São Paulo: Novatec, 2012.

SILVA, Maurício Samy. HTML 5: a linguagem de marcação que revolucionou a web. São Paulo: Novatec, 2011.

NIEDERAUER, Juliano. Desenvolvendo websites com PHP: aprenda a criar websites dinâmicos e interativos com PHP e bancos de dados. 2. ed. rev. e atual. São Paulo: Novatec, 2011.

## Referências Complementares

FLANAGAN, David. **o guia definitivo**. . O Really. 2012

RESIG, John. **Secrets of the JavaScript Ninja**. Manning. 2ed. 2016.

SILVA, Maurício Samy. **Criando sites com HTML: sites de alta qualidade com HTML e CSS**. . Novatec. 2010

SOARES, Walace. **PHP 5: conceitos, programação e integração com banco de dados**. . Érica. 2010

DALL'OGLIO, Pablo. **PHP: programando com orientação a objetos**. . Novatec. 2009

DEITEL, Paul J. Ajax,. **Rich Internet applications e desenvolvimento Web para programadores**. . Pearson Prentice Hall. 2009

IEPSEN, Edécio Fernandes. **Lógica de Programação e Algoritmos com JavaScript.** Novatec. 2018.

## Referências na Internet

https://www.w3schools.com

https://developer.mozilla.org/pt-BR/docs/Web

https://illustrated.dev/advancedjs